# Amazon Performance Sales Data Cleansing in BigQuery

SQL scripts and decision log. Project: Amazon-Data-Analytics | Dataset: cleansing_table | Source table: amazon_first_raw_data (raw)

## 1. Overview

This document records every cleansing step applied to the Amazon Sales dataset, in order, from the raw table to the final table. Each table is created from the previous one, so the raw data is never modified and the whole process can be repeated.

| Table | Purpose |
|---|---|
| amazon_first_raw_data | Raw data as uploaded (1,465 rows, 1,351 unique product_id) |
| amazon_clean_data | After removing duplicate product_id (1,351 rows) |
| amazon_clean_data_v2 | Rating converted to FLOAT64 |
| amazon_category | Category split into three columns |
| amazon_discount_percentage | discount_percentage rescaled to 0-100 |
| amazon_user | user_id and review_id split into one row per review (10,598 rows) |
| amazon_final_data | Final cleaned table |

## 2. Known Limitations

- user_name, review_title, and review_content have not been split per review. They still contain the combined text and repeat on every row produced by the split.
- 9 products whose user_id, review_id, and user_name counts did not align lost some of their reviews during the split (the products themselves are kept).
- review_id alone is not unique. The unique key is the combination of product_id and review_id.

## 3. Step-by-Step Scripts

### Step 1: Data profiling

Check row count, unique products, NULLs, invalid ratings, and price/discount formats before changing anything.

```
--Profiling data and checking the formats n the number of column
SELECT
  COUNT(*) AS total_baris,
  COUNT(DISTINCT product_id) AS produk_unik,
  COUNTIF(product_id IS NULL) AS id_kosong,
  COUNTIF(rating_count IS NULL) AS rating_count_kosong,
  COUNTIF(about_product IS NULL) AS about_kosong,
  COUNTIF(product_name != TRIM(product_name)) AS nama_ada_spasi,
  COUNTIF(NOT STARTS_WITH(product_link, 'http')) AS link_aneh
FROM `amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data`;
```

```sql
-- Invalid rating values
SELECT rating, COUNT(*) AS jumlah
FROM `amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data`
WHERE SAFE_CAST(rating AS FLOAT64) IS NULL OR SAFE_CAST(rating AS FLOAT64) NOT
BETWEEN 1 AND 5
GROUP BY rating;
```

> **Decision:** Results: 1,465 rows, 1,351 unique products, 2 empty rating_count, 1 invalid rating (value "|").

### Step 2: Investigate duplicates

```sql
--Product IDs that appear more than once
SELECT product_id, COUNT(*) AS jumlah_produk
FROM `amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data`
GROUP BY product_id
HAVING COUNT(*) > 1
ORDER BY jumlah_produk DESC;
```

```sql
--Checking there fully identical rows
SELECT
  COUNT(*) AS total_baris,
  COUNT(DISTINCT TO_JSON_STRING(t)) AS baris_unik_penuh
FROM `amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data` t;
```

> **Decision:** All 1,465 rows are unique when every column is compared, but duplicated product_id rows are identical in every column except img_link and product_link. They are business duplicates and safe to remove.

### Step 3: Remove duplicate product_id

```sql
CREATE OR REPLACE TABLE `amazon-data-analytics-
511402.cleansing_table.amazon_clean_data` AS
SELECT * EXCEPT(rn) FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY product_id ORDER BY product_link) AS rn
  FROM `amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data`
)
WHERE rn = 1;
```

```sql
-- Validation: both numbers must be 1351
SELECT COUNT(*) AS total, COUNT(DISTINCT product_id) AS unik
FROM `amazon-data-analytics-511402.cleansing_table.amazon_clean_data`;
```

> **Decision:** 114 duplicate product_id rows removed (1,465 to 1,351). Other columns were identical; only the URLs differed.

### Step 4: Fix the rating column

```sql
-- Inspect the problem row
SELECT product_id, product_name, rating, rating_count
FROM amazon-data-analytics-511402.cleansing_table.amazon_clean_data
WHERE SAFE_CAST(rating AS FLOAT64) IS NULL;

CREATE OR REPLACE TABLE amazon-data-analytics-
511402.cleansing_table.amazon_clean_data_v2 AS
```

```sql
SELECT * REPLACE (SAFE_CAST(rating AS FLOAT64) AS rating)
FROM amazon-data-analytics-511402.cleansing_table.amazon_clean_data;

--validation of rating format
SELECT
  COUNTIF(rating IS NULL) AS rating_null,
  MIN(rating) AS rating_min,
  MAX(rating) AS rating_max
FROM amazon-data-analytics-511402.cleansing_table.amazon_clean_data_v2;
```

> **Decision:** One invalid rating ("|") converted to NULL with SAFE_CAST; the row is kept. Result: rating_null = 1, min = 2, max = 5. The 2 empty rating_count values were left as NULL (filling with 0 would be misleading).

### Step 5: Split the category column

```sql
CREATE OR REPLACE TABLE amazon-data-analytics-
511402.cleansing_table.amazon_category AS
SELECT
  * EXCEPT(category),
  category AS category_full,
  SPLIT(category, '|')[SAFE_OFFSET(0)] AS main_category,
  SPLIT(category, '|')[SAFE_OFFSET(1)] AS sub_category,
  SPLIT(category,'|')[SAFE_OFFSET(2)] AS sub_category_2,
  SPLIT(category,'|')[SAFE_OFFSET(3)] AS sub_category_3
FROM amazon-data-analytics-511402.cleansing_table.amazon_clean_data_v2;

--validation of split category
SELECT main_category, COUNT(*) AS jumlah
FROM amazon-data-analytics-511402.cleansing_table.amazon_category
GROUP BY main_category
ORDER BY jumlah DESC;
```

> **Decision:** category split into main_category, sub_category, and sub_category_2 with SPLIT. No NULL values in main_category. (In the working tables the first column was named category_utama; rename it here if you want the script to match your tables exactly.)

### Step 6: Validate and standardize prices and discounts

```sql
SELECT
  COUNTIF(discounted_price <= 0 OR actual_price <= 0) AS harga_nol_negatif,
  COUNTIF(discounted_price > actual_price) AS diskon_lebih_mahal,
  COUNTIF(discount_percentage NOT BETWEEN 0 AND 100) AS diskon_diluar_range,
  COUNTIF(ABS(discount_percentage * 100 - ROUND((actual_price - discounted_price) /
actual_price * 100)) > 1) AS diskon_tidak_cocok
FROM amazon-data-analytics-511402.cleansing_table.amazon_category;
```

The first run returned 0 for every check except discount_mismatch (1,304 rows). The cause was a scale difference: discount_percentage was stored as a fraction (0.64) while the calculated value was a percentage (64).

```sql
CREATE OR REPLACE TABLE amazon-data-analytics-
511402.cleansing_table.amazon_discount_percentage AS
SELECT * REPLACE (ROUND(discount_percentage * 100) AS discount_percentage)
FROM amazon-data-analytics-511402.cleansing_table.amazon_category;
-- Re-run the validation query above on amazon_discount_percentage
```

> **Decision:** discount_percentage rescaled from a 0-1 fraction to a 0-100 percentage (multiplied by 100). After the change, all four checks returned 0.

### Step 7: Split user_id and review_id

Check first whether the number of items in each multi-value cell lines up across columns.

```
--Check the number of user in one column
SELECT product_id,
  ARRAY_LENGTH(SPLIT(user_id, ',')) AS jumlah_user_id,
  ARRAY_LENGTH(SPLIT(user_name, ',')) AS jumlah_user_name,
  ARRAY_LENGTH(SPLIT(review_id, ',')) AS jumlah_review_id
FROM amazon-data-analytics-511402.cleansing_table.amazon_discount_percentage;

--Check the different of row
SELECT COUNT(*) AS misaligned_rows
FROM amazon-data-analytics-511402.cleansing_table.amazon_discount_percentage
WHERE ARRAY_LENGTH(SPLIT(user_id, ',')) != ARRAY_LENGTH(SPLIT(review_id, ',')) OR
  ARRAY_LENGTH(SPLIT(user_id, ',')) != ARRAY_LENGTH(SPLIT(user_name, ','));
```

Result: 9 of 1,351 rows were misaligned (likely commas inside user_name). Only user_id and review_id, which never contain commas, were split. Pairs are matched by position with WITH OFFSET.

```
--Create table to clasify the user
CREATE OR REPLACE TABLE amazon-data-analytics-511402.cleansing_table.amazon_user AS
SELECT * EXCEPT (user_id, review_id),
  TRIM(uid) AS user_id,
  TRIM(rid) AS review_id
FROM amazon-data-analytics-511402.cleansing_table.amazon_discount_percentage,
UNNEST(SPLIT(user_id, ',')) AS uid WITH OFFSET AS pos1,
UNNEST(SPLIT(review_id, ',')) AS rid WITH OFFSET AS pos2
WHERE pos1 = pos2;

--Validation of Classification
SELECT COUNT(*) AS total_baris, COUNT(DISTINCT review_id) AS review_unik
FROM amazon-data-analytics-511402.cleansing_table.amazon_user;

--Check the difference between different products n unique review pairs
SELECT review_id,
  COUNT(*) AS jumlah,
  COUNT(DISTINCT product_id) AS produk_berbeda
FROM amazon-data-analytics-511402.cleansing_table.amazon_user
GROUP BY review_id
HAVING COUNT(*) > 1
ORDER BY jumlah DESC;

SELECT COUNT(*) AS total,
  COUNT(DISTINCT CONCAT(product_id, '|', review_id)) AS unique_review_pairs
FROM amazon-data-analytics-511402.cleansing_table.amazon_user;
```

> **Decision:** Split produced 10,598 rows. review_id alone is not unique (9,265 distinct) because the same review appears under several products, but the product_id + review_id pair is unique (10,598), so no rows were removed.

### Step 8: Text and link checks

```
--Checking the text formats
```

```sql
SELECT
  COUNTIF(product_name IS NULL OR TRIM(product_name) = '') AS empty_name,
  COUNTIF(about_product IS NULL OR TRIM(about_product) = '') AS empty_about,
  COUNTIF(review_content IS NULL OR TRIM(review_content) = '') AS empty_review,
  COUNTIF(NOT STARTS_WITH(img_link, 'http')) AS another_img,
  COUNTIF(NOT STARTS_WITH(product_link, 'http')) AS another_link,
  COUNTIF(product_name != TRIM(product_name)) AS space_name
FROM amazon-data-analytics-511402.cleansing_table.amazon_user;
```

> **Decision:** All checks returned 0. No further text cleaning was needed.

### Step 9: Create the final table and compare with raw data

```sql
--Create final table
CREATE OR REPLACE TABLE amazon-data-analytics-
511402.cleansing_table.amazon_final_data AS
SELECT *
FROM amazon-data-analytics-511402.cleansing_table.amazon_user;

--Compare the raw data
SELECT 'raw' AS raw_table, COUNT(*) AS row_data, COUNT(DISTINCT product_id) AS
unique_product
FROM amazon-data-analytics-511402.cleansing_table.amazon_first_raw_data
UNION ALL
SELECT 'final' AS final_table, COUNT(*), COUNT(DISTINCT product_id)
FROM amazon-data-analytics-511402.cleansing_table.amazon_final_data;

--Check data type
SELECT column_name, data_type
FROM amazon-data-analytics-511402.cleansing_table.INFORMATION_SCHEMA.COLUMNS
WHERE table_name = 'amazon_final_data';
```

> **Decision:** Raw: 1,465 rows / 1,351 unique products. Final: 10,598 rows / 1,351 unique products. No product was lost. Data types were verified as correct.

## 4. Summary of Decisions

- 114 duplicate product_id rows removed (other columns identical; only URLs differed).
- One invalid rating ("|") converted to NULL using SAFE_CAST; the row was kept.
- Two empty rating_count values left as NULL.
- category split into main_category, sub_category, and sub_category_2.
- discount_percentage rescaled from 0-1 to 0-100; validated against prices.
- user_id and review_id split with UNNEST into 10,598 rows; product_id + review_id is the unique key.
- Text and link columns passed all checks without changes.

## 5. Suggested Next Steps

- Split user_name, review_title, and review_content per review for the 1,342 aligned products.
- Run simple analyses to see the cleansing at work: average rating per main_category, products with the largest discounts.